

Uart_Esp32 - Projektdokumentation - Firmware

Projektname: Uart_Esp32
Dokumentenart: Projektdokumentation
Thema: Firmware fuer ESP32-WROOM-32 / NodeMCU-32S
Erstellungsdatum: 2026-09-17
Autor / verantwortliche Person: OpenAI ChatGPT (technische Erstellung)
Version: v0.5
Status: Entwurf
Aenderungsdatum: 2026-09-17
Ablageort: Projektordner Uart_Esp32/docs
Referenzen: Regeln_Projektdokumentation_PDF_DOCX_Pflicht_(5).pdf; Projektstand v0.5; lokaler PlatformIO-Buildfehler in markWifiConnected()

Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.1	2026-09-17	OpenAI ChatGPT	Entwurf	Reduzierter WROOM32-Projektstand erstellt
v0.2	2026-09-17	OpenAI ChatGPT	Entwurf	Projekt in Uart_Esp32 umbenannt und Dokumentationsregeln umgesetzt
v0.3	2026-09-17	OpenAI ChatGPT	Entwurf	Buildfehler durch typensichere Integer-Vergleiche behoben
v0.4	2026-09-17	OpenAI ChatGPT	Entwurf	Arduino-Secrets, NVS-Fallbacks und WLAN-Provisionierung ergaenzt
v0.5	2026-09-17	OpenAI ChatGPT	Entwurf	WLAN-State-/ Fallback-Logik robust gemacht; Secrets als primaerer WLAN-Kandidat; Recovery-AP, DHCP-Fallback, Resetdiagnose und Netzwerkstatus ergaenzt
v0.6	2026-09-17	OpenAI ChatGPT	Entwurf	Compilerfehler in markWifiConnected() behoben; Versionsstand aktualisiert

Ziel des Standes v0.6

Der Stand v0.6 behebt einen C++-Syntaxfehler in markWifiConnected(), der den lokalen PlatformIO-Build mit "else without a previous if" abbrach. Ursache war eine Zuweisung zwischen einem if und dem zugehoerigen

else if. Die WLAN-Fallback-Architektur aus v0.5 bleibt unverändert erhalten. Die Quellenwahl ist jetzt korrekt geklammert; die DHCP-Fallback-Markierung wird danach separat gesetzt.

Die Datei `arduino_secrets.h` bleibt optional und wird nicht mit Git versioniert. Wenn dort gueltige WLAN-Zugangsdaten vorhanden sind, werden sie standardmaessig als erster WLAN-Kandidat verwendet. Ein ueber das Webinterface gespeichertes NVS-WLAN bleibt als unabhaengiger zweiter Kandidat erhalten.

WLAN-Fallback-Kette

Standardreihenfolge beim Start:

1. WLAN aus `arduino_secrets.h`, sofern die SSID gueltig und kein Platzhalter ist.
2. Separat in NVS gespeichertes WLAN, sofern vorhanden und nicht identisch mit dem Secret-WLAN.
3. Wenn eine statische IP konfiguriert ist und alle Kandidaten scheitern: Wiederholung der Client-Versuche mit DHCP.
4. Wenn keine Client-Verbindung zustande kommt: Fallback-AP im Modus AP+STA.
5. Wenn AP+STA nicht aktiviert werden kann: reiner AP-Recovery-Modus.
6. Wenn der konfigurierte AP selbst nicht startet: offener Emergency-AP `Uart_Esp32-Recovery-<ChipID>`.

Der reine AP-Recovery-Modus wird bewusst nicht durch periodische STA-Umschaltungen unterbrochen. Nach einer im Webinterface gespeicherten WLAN-Konfiguration startet das Geraet neu und versucht die Client-Verbindung erneut.

Arduino-Secrets und NVS

Fuer normale Laufzeitparameter bleibt die Hierarchie:

1. NVS-Wert, wenn der Key vorhanden ist.
2. Wert aus `arduino_secrets.h` / Compile-Default.
3. Interner sicherer Default.

Fuer WLAN-Zugangsdaten gilt standardmaessig eine andere Logik: `arduino_secrets.h` ist der primaere Verbindungskandidat und NVS der Fallback. Diese Reihenfolge kann ueber `UART_WIFI_PREFER_SECRETS` umgedreht werden.

Platzhalter wie `YOUR_WIFI_SSID`, `YOUR_MQTT_USER` oder `CHANGE_ME` werden von der Hilfsfunktion `uartSecretTextUsable()` als nicht konfiguriert behandelt.

WLAN-Fallback-Schalter

```
#define UART_WIFI_PREFER_SECRETS    true
#define UART_WIFI_ALLOW_NVS_FALLBACK true
#define UART_WIFI_ALLOW_DHCP_FALLBACK true
#define UART_WIFI_ALLOW_EMERGENCY_AP true
#define UART_AP_KEEP_AFTER_CONNECT  false
```

Weitere Zeitwerte koennen bei Bedarf in `ConfigDefaults.h` ueberschrieben werden: Verbindungs-Timeout, Reconnect-Intervall, AP-Startverzoegerung und AP-Abschaltverzoegerung.

AP-/STA-Verhalten

Der Fallback-AP wird bevorzugt als AP+STA gestartet. Damit bleibt die Konfigurationsseite erreichbar, waehrend die Firmware im Hintergrund spaeter erneut die vorhandenen WLAN-Kandidaten testen kann. Nach erfolgreicher STA-Verbindung bleibt der AP standardmaessig noch 15 Sekunden aktiv und wird danach abgeschaltet. Bei einem spaeteren Verbindungsverlust wird er nach kurzer Wartezeit wieder aktiviert.

Der Moduswechsel wird ueber `setWifiModeRobust()` mit mehreren Versuchen und einem kontrollierten WIFI_OFF-Recovery ausgefuehrt. Dadurch wird vermieden, dass bei einem fehlerhaften Moduswechsel unmittelbar `softAP()` aufgerufen wird, obwohl der AP-Modus noch nicht aktiv ist.

Netzwerk-Webseite

`/network` zeigt jetzt zusaetzlich:

- aktuellen WLAN-Modus (OFF, STA, AP, AP+STA),
- Verbindungszustand,
- aktive WLAN-Quelle (`arduino_secrets.h` oder NVS),
- aktuelle SSID/IP/RSSI bei Verbindung,
- AP-SSID und AP-IP,
- Reconnect- und Fehlerzaehler,
- erkannte Secret-SSID ohne Passwort,
- gespeicherte NVS-SSID ohne Passwort.

Die Route `/clear_wifi_nvs` loescht nur das gespeicherte NVS-WLAN. Eine vorhandene Secret-Konfiguration bleibt davon unberuehrt.

Reset- und Builddiagnose

Beim Boot werden jetzt Firmwarestand, Build-Zeit und Resetgrund ausgegeben. Die Hauptseite zeigt den Resetgrund ebenfalls an. Typische Werte sind `POWERON_RESET`, `SOFTWARE_RESET`, `TASK_WDT_RESET`, `BROWNOUT_RESET` oder `DEEPSLEEP_RESET`.

Beispiel:

```
Uart_Esp32 v0.6
[BOOT] Build: Sep 17 2026 15:30:00
[BOOT] Resetgrund: POWERON_RESET (1)
[CONFIG] arduino_secrets.h: vorhanden
[CONFIG] WLAN Primaer: arduino_secrets.h (MeinWLAN)
[CONFIG] WLAN Fallback: NVS (AlternativWLAN)
[WIFI] Verbindungsversuch 1: MeinWLAN (arduino_secrets.h)
[WIFI] Verbunden: 192.168.24.170, SSID MeinWLAN, Quelle arduino_secrets.h, RSSI -55 dBm
```

Projektstruktur

```
Uart_Esp32/
.gitignore
platformio.ini
README.md
include/
  arduino_secrets.example.h
  arduino_secrets.h      # lokal, nicht in Git
src/
  ConfigDefaults.h
  BatteryMonitor.cpp/.h
  DeepSleepManager.cpp/.h
  MqttManager.cpp/.h
  OtaManager.cpp/.h
  main.cpp
docs/
  2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.6.*
  2026-09-17 - Uart_Esp32 - Commit-Nachricht - Buildfix-markWifiConnected - v0.6.txt
```

Web-Routen

Route	Funktion
/	Hauptstatus inkl. Firmware-/Reset-/WLAN-Status
/network	WLAN-Status, Scan/Auswahl, NVS-Fallback, AP, Hostname, NTP
/wifi_rescan	WLAN-Scan neu starten, sofern STA/AP+STA verfuegbar
/clear_wifi_nvs	gespeichertes NVS-WLAN loeschen, POST
/save_network	NVS-Fallback und Netzwerkeinstellungen speichern, Neustart
/mqtt	MQTT-Einstellungen / Verbindungstest
/battery	ADC-Batteriemessung
/deepsleep	Deep-Sleep-Konfiguration
/ota	Browser OTA, ArduinoOTA, Pull OTA
/reboot	Neustart, POST

Geaenderte Dateien in v0.6

- README.md
- include/arduino_secrets.example.h
- src/ConfigDefaults.h
- src/main.cpp
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.6.md
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.6.docx
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.6.pdf
- docs/2026-09-17 - Uart_Esp32 - Commit-Nachricht - Buildfix-markWifiConnected - v0.6.txt

Pruefung und offene Verifikation

Der konkrete Compilerfehler aus dem lokalen PlatformIO-Build wurde im Quelltext korrigiert. Statisch wurde geprueft, dass das else if wieder unmittelbar zum if gehoert und keine Anweisung mehr dazwischen liegt. Da in der Erstellungsumgebung kein vollstaendiger PlatformIO-ESP32-Build verfuegbar ist, bleibt der lokale Build der verbindliche Verifikationstest:

```
pio run -e uart_esp32_usb -t clean
pio run -e uart_esp32_usb
pio run -e uart_esp32_usb -t upload
```

Danach sollte der serielle Monitor insbesondere zeigen, welche WLAN-Quelle verwendet wird und ob ein DHCP-/AP-Fallback aktiviert wurde.

Auslieferungsform

Bereitgestellt werden ein vollstaendiges ZIP mit Root-Ordner Uart_Esp32, ein ZIP mit nur geaenderten/neuen Dateien in derselben Struktur, eine Patch-Datei, eine englische Commit-Nachricht mit Datum und Dateiliste sowie Projektdokumentation als PDF und bearbeitbare DOCX mit identischem Stand v0.6.